

Reflective Journal: Building a CNN for MNIST Handwritten Digit

Classification

Author: Group Project (Cherluk, Michael, Olu, Taki)

Course: ITAI 2676

Learning Insights

What new concepts about CNNs did you discover during this lab?

Working through this lab made the internal mechanisms of Convolutional Neural Networks (CNNs) significantly clearer. Implementing the Conv2D and MaxPooling2D layers step-by-step allowed us to understand how a network systematically breaks down visual information. We saw how convolutional layers act as feature detectors, scanning small sections of an image with 3x3 kernels to identify edges and curves, while pooling layers retain the strongest activations and discard unnecessary spatial data to reduce computational load. We also understood how the Flatten layer serves as the essential bridge, unrolling multi-dimensional feature maps into a 1D vector so the Dense layer can calculate final class probabilities.

How does this lab connect to your previous knowledge of neural networks?

Previously, neural networks were viewed largely as generic mathematical graphs. This lab bridged abstract fully connected networks with spatial computer vision architectures. While standard Dense layers look at every input feature uniformly, Conv2D layers leverage local connectivity and weight sharing, making them specifically tailored for image processing before passing the extracted features into traditional dense classification layers.

What surprised you about working with CNNs and the MNIST dataset?

What surprised us most was how complex even simple 28x28 grayscale images are for a machine to interpret. To a human, a handwritten '7' is instantly recognizable, but to the network, it is just a matrix of pixel intensities. We were surprised by how effectively a relatively compact architecture—using just 32 and 64 filters—could decipher these pixel matrices. Furthermore, the necessity of preprocessing (normalizing pixel values to a 0.0–1.0 range and one-hot encoding labels) highlighted that the model's success depends just as much on data preparation as on architecture.

Challenges and Growth

What specific challenges did you encounter while implementing the CNN?

A real challenge encountered during this project was grasping tensor transformations and how the shape of data changes as it moves through the network. Initially, picturing how the 28x28x1 image tensor was morphed—specifically why the Flatten layer is needed before the Dense layer—was a stumbling block.

How did you overcome these challenges?

We overcame this by mapping out exact dimensions at each stage, tracking how the first

pooling layer reduced 26x26 output to 13x13, and the second pooling layer reduced it to 5x5. Calculating the exact matrix dimensions ($5 \times 5 \times 64 = 1600$) allowed us to visualize the architecture perfectly and understand why the Flatten layer outputs a 1,600-value vector.

What resources or strategies helped you understand difficult concepts?

Stepping through code cells sequentially, examining shape printouts (`model.summary()`), and analyzing side-by-side training and validation accuracy curves helped solidify our understanding of how hyperparameters like batch size, epochs, and dropout affect training dynamics.

Personal Development

How has this lab changed your understanding of deep learning?

This laboratory completely changed our view of deep learning. We no longer see it as a 'black box' that magically outputs answers, but rather as a logical, step-by-step visual processing pipeline.

What aspects of CNNs would you like to explore further?

Moving forward, we want to investigate how CNNs scale beyond simple digits. We are eager to explore transfer learning with larger architectures like VGG16 and dive into object detection models, applying these foundational concepts to more complex, real-world computer vision tasks.

If you were already familiar with CNNs, what new perspectives did you gain?

For team members with prior familiarity, seeing the direct impact of the 50% Dropout layer on validation loss and observing healthy generalization without overfitting reinforced the practical importance of regularization techniques in production-grade neural networks.